

EJERCICIO UNIDAD 1

Informe Técnico

PGE

8 DE SEPTIEMBRE DE 2026

Profesora:

- Bulacio, Julia.

Equipo:

- Bianconi, Clara.
- Guzmán Domínguez, Lucas Santiago.
- Roldan, Tiago.

Índice

Introducción	2
Objetivo general	3
Alcance	3
Requerimientos funcionales	4
Requerimientos no funcionales	4
Análisis de la página seleccionada	5
Problemas identificados.....	6
Propuestas de mejora.....	6
Prototipo desarrollado.....	7
Conclusión	8
Anexos.....	9

Introducción

Por medio del presente informe se pretende explicar en detalle el proceso de análisis, diseño y desarrollo de un prototipo orientado a la mejora de la interfaz de usuario de la página web “jobboard.universia.net” haciendo uso del lenguaje de programación C++ y de los diferentes conceptos y herramientas estudiados durante el desarrollo de la materia.

El proyecto parte del análisis de “jobboard.universia.net” una aplicación web de uso habitual para aquellas personas que buscan conseguir empleo; con el propósito de identificar aquellas características de su interfaz que puedan resultar poco intuitivas, incómodas o difíciles de utilizar. A partir de los problemas de usabilidad detectados, se busca plantear diferentes alternativas que permitan mejorar la experiencia del usuario y demostrar su funcionamiento mediante la creación de un prototipo funcional.

Para llevar a cabo estas mejoras se aplicarán diferentes conceptos estudiados durante el curso, buscando un uso conjunto de estas herramientas, permitiendo desarrollar una interfaz capaz de responder dinámicamente a las acciones realizadas por el usuario, gestionar correctamente las interacciones continuas con el programa, modificar elementos visuales en tiempo real y adaptar los diferentes tipos o formatos de datos utilizados por la aplicación.

El prototipo desarrollado no pretende reemplazar completamente al software seleccionado ni reproducir la totalidad de sus funcionalidades. En cambio, busca representar aquellas secciones de la interfaz relacionadas directamente con los problemas identificados y demostrar, de manera práctica, cómo determinadas modificaciones pueden contribuir a obtener una experiencia de usuario más sencilla, intuitiva y eficiente.

Finalmente, mediante el desarrollo del proyecto se pretende no solo aplicar los conocimientos adquiridos sobre programación en C++, sino también comprender la importancia que posee el diseño de interfaces dentro del desarrollo de software, considerando que la funcionalidad de un sistema debe estar acompañada por mecanismos de interacción claros y accesibles para sus usuarios.

Objetivo general

El objetivo general del proyecto es analizar la interfaz de usuario de una aplicación de software e identificar diferentes problemas de usabilidad presentes en ella, con el propósito de diseñar e implementar un prototipo funcional en C++ que demuestre posibles soluciones a dichos problemas.

Para ello, se busca aplicar los diferentes conocimientos adquiridos durante el desarrollo de la materia, integrándolos dentro de una misma solución que permita mejorar la interacción entre el usuario y el sistema.

Asimismo, el proyecto busca fortalecer los conocimientos relacionados con el desarrollo de interfaces gráficas y la programación orientada a eventos, comprendiendo la importancia de diseñar aplicaciones que no solamente cumplan con sus funciones internas, sino que además proporcionen al usuario una experiencia clara, intuitiva y eficiente.

Alcance

El presente proyecto se encuentra enfocado en el análisis y mejora de la usabilidad de una aplicación de software existente, tomando como principal objeto de estudio aquellos elementos de su interfaz que puedan generar dificultades durante la interacción con el usuario.

El análisis estará limitado principalmente a las características relacionadas con la experiencia de usuario y la interfaz gráfica. Por lo tanto, no se pretende realizar una reconstrucción completa de la aplicación seleccionada ni modificar directamente su código fuente original. En cambio, se identificarán determinadas funcionalidades o mecanismos de interacción que presenten problemas de usabilidad y se desarrollará una propuesta alternativa que permita demostrar cómo podrían ser mejorados.

A partir de los problemas encontrados, se desarrollará un prototipo funcional en C++ que represente las principales soluciones propuestas. Tal como se mencionó con anterioridad, dicho prototipo no deberá constituir necesariamente un programa completo, sino demostrar de forma clara cómo las modificaciones implementadas permiten solucionar o reducir los problemas de usabilidad previamente identificados.

El desarrollo incluirá la implementación de los cuatro conceptos principales vistos a lo largo del curso: "Callbacks", bucles, eventos "Paint" y funciones de conversión. Estos elementos serán utilizados de acuerdo con las necesidades particulares de la solución, buscando que cada uno tenga una participación identificable dentro del funcionamiento del prototipo.

De esta manera, el resultado final del proyecto estará compuesto por un prototipo funcional desarrollado en C++, buscando demostrar tanto la correcta aplicación de los contenidos estudiados durante la materia como la capacidad de analizar una problemática de usabilidad y desarrollar una solución concreta para mejorar la experiencia de todo aquel usuario que utilice la página.

Requerimientos funcionales

Como bien se mencionó en la introducción, nuestro proyecto busca analizar diferentes problemas presentes en la interfaz de usuario de una aplicación de software y proponer soluciones que permitan simplificar y mejorar la interacción entre el usuario y el sistema.

El sistema por desarrollar no busca reproducir completamente la aplicación seleccionada, sino enfocarse específicamente en aquellos aspectos de su interfaz que hayan sido identificados como poco intuitivos, incómodos o difíciles de utilizar.

Por ello, los principales requerimientos funcionales de nuestro proyecto son:

- El sistema debe permitir representar mediante un prototipo funcional las secciones de la aplicación seleccionada que presenten los problemas de usabilidad identificados, sin una reproducción completa de sus funcionalidades.
- El sistema debe permitir al usuario interactuar con los diferentes elementos implementados en la interfaz, tales como botones, opciones, campos u otros controles necesarios para demostrar las mejoras propuestas.
- El sistema debe proporcionar retroalimentación visual ante determinadas interacciones del usuario, permitiendo reconocer claramente cuándo una acción fue realizada, seleccionada o procesada.
- Las funcionalidades implementadas deben estar directamente relacionadas con uno o más de los problemas de usabilidad previamente identificados, de manera que sea posible demostrar cómo las modificaciones propuestas contribuyen a mejorar la interacción con el software.

Requerimientos no funcionales

En el contexto de este proyecto, los requerimientos no funcionales describen las diferentes características y propiedades que deberá presentar el prototipo para que las mejoras desarrolladas puedan ser utilizadas y comprendidas correctamente.

Por ello, los principales requerimientos no funcionales de nuestro proyecto son:

- La interfaz del prototipo debe ser intuitiva y sencilla de utilizar, procurando que un usuario pueda comprender las principales funcionalidades implementadas sin requerir conocimientos técnicos avanzados.
- Los elementos visuales de la interfaz deben mantener una distribución clara y coherente, evitando agregar componentes innecesarios que puedan dificultar la navegación o comprensión del sistema.
- El prototipo debe proporcionar una respuesta visual clara ante las acciones realizadas por el usuario, evitando situaciones en las que este no pueda determinar si una acción fue correctamente reconocida por el programa.
- Las modificaciones visuales deben producirse de manera fluida y comprensible, procurando evitar cambios innecesarios que puedan afectar negativamente el uso de la interfaz.

- El código fuente desarrollado en C++ debe mantener una estructura clara, organizada y comprensible, separando adecuadamente las diferentes responsabilidades y funcionalidades implementadas.
- El prototipo debe poseer la estabilidad suficiente para demostrar las funcionalidades desarrolladas, evitando errores que impidan realizar las interacciones principales necesarias para comprobar las mejoras propuestas.
- La interfaz debe mantener una coherencia visual entre sus diferentes elementos, procurando utilizar tamaños, posiciones, textos y mecanismos de interacción consistentes durante la utilización del prototipo.
- El prototipo debe mantenerse dentro de un alcance reducido y demostrativo, priorizando la correcta implementación y demostración de las mejoras de usabilidad por encima del desarrollo de funcionalidades adicionales que no estén relacionadas directamente con los objetivos del proyecto.

Análisis de la página seleccionada

Para el desarrollo del proyecto se seleccionó como objeto de análisis el portal de empleos de Universia correspondiente a la Universidad Blas Pascal (UBP). La función principal de esta plataforma consiste en permitir que estudiantes, graduados y demás usuarios interesados puedan consultar diferentes ofertas laborales disponibles y acceder a información relacionada con cada puesto de trabajo.

Luego de analizar su interfaz, se decidió centrar el proyecto específicamente en la presentación y organización del sistema de filtros. La elección de este elemento se debe a que los filtros constituyen una parte importante de la interacción dentro de una plataforma de búsqueda de empleo, ya que permiten organizar una gran cantidad de información y facilitar la localización de publicaciones relevantes.¹

Si bien estas funcionalidades cumplen con su propósito básico, consideramos que su funcionamiento y organización pueden resultar poco intuitivos, o incluso tediosos de utilizar, especialmente teniendo en cuenta que la búsqueda y comparación de ofertas laborales constituyen algunas de las interacciones más importantes dentro de una plataforma de estas características.

Por esta razón, el prototipo desarrollado se enfocará principalmente en estudiar conceptualmente cómo se presentan y agrupan las opciones de filtrado y cómo interactúa el usuario con el panel que las contiene.

¹ Ver Anexo 1.

Problemas identificados

Como explicamos en el punto anterior, uno de los principales puntos débiles identificados corresponde al sistema utilizado para filtrar las ofertas laborales.

Los filtros constituyen una herramienta fundamental dentro de una bolsa de trabajo, debido a que permiten reducir una cantidad potencialmente elevada de publicaciones hasta obtener aquellas que realmente puedan resultar relevantes para el usuario. Una persona puede, por ejemplo, estar interesada únicamente en determinadas áreas laborales, ubicaciones o modalidades de trabajo.

Por este motivo, consideramos que el filtrado no debería presentarse simplemente como una funcionalidad secundaria, sino como uno de los principales mecanismos de navegación de la plataforma.

No obstante, debido al alcance del prototipo, no se analizaron ni implementaron problemas relacionados con el funcionamiento interno del algoritmo de búsqueda, la precisión de los resultados, la selección efectiva de filtros, la paginación o el contenido de las ofertas laborales. Estos aspectos exceden las funcionalidades desarrolladas en el código entregado. De esta manera, el problema queda limitado a una situación concreta y demostrable: Representar un panel de filtros de manera organizada, y gestionar su apertura, cierre, y actualización ante las acciones del usuario

Propuestas de mejora

La propuesta de mejora consiste en representar los filtros mediante una ventana o panel independiente, accesible mediante un botón claramente identificado como "Filtros". De esta forma, el usuario puede decidir cuándo visualizar las opciones disponibles y cuándo ocultarlas para regresar al contenido principal.²

Dentro de este panel, los filtros se encuentran organizados mediante grupos claramente diferenciados, donde cada grupo posee un título y una colección de opciones. Esta organización busca demostrar una alternativa más estructurada para presentar los criterios de búsqueda y facilitar su identificación por parte del usuario.

La interacción principal propuesta consiste en permitir abrir y cerrar el panel de filtros, además de marcar y desmarcar las diferentes opciones disponibles. El estado de cada opción seleccionada se almacena mientras el programa permanece abierto, permitiendo que la representación visual del panel permanezca sincronizada con el estado interno de la aplicación.

Cuando se produce una interacción que modifica la interfaz, como la apertura del panel o la selección de una opción, se solicita un nuevo pintado de la ventana. De esta manera, los cambios realizados por el usuario se reflejan inmediatamente en pantalla. El panel también puede cerrarse mediante la cruz ubicada en su esquina, mediante el botón "Cerrar" o haciendo clic fuera de la ventana emergente.

Con esto en mente, se reitera que la propuesta implementada no contempla todavía la aplicación efectiva de los filtros sobre ofertas laborales. Las opciones pueden seleccionarse y deseleccionarse visualmente, pero su estado no modifica las ofertas presentadas ni se

² Ver anexo 2.

encuentra conectado con una base de datos. Esta limitación es intencional, ya que el objetivo del prototipo es demostrar principalmente la interacción con el panel y los conceptos de programación orientada a eventos solicitados por la consigna.

En consecuencia, la mejora propuesta debe entenderse como un prototipo funcional de interacción y organización de filtros, y no como una reproducción completa del funcionamiento del portal de empleos. Su alcance se concentra en demostrar una alternativa de interfaz más organizada y los mecanismos técnicos necesarios para responder dinámicamente a las acciones realizadas por el usuario.

Prototipo desarrollado

Para demostrar la propuesta se desarrolló un prototipo en C++ mediante una interfaz gráfica nativa de Windows, utilizando Win32 y GDI. La aplicación representa de forma simplificada el portal de empleos seleccionado, incluyendo elementos visuales como encabezado, buscador, ofertas laborales y un botón "Filtros" que permite acceder a una ventana emergente con las diferentes opciones disponibles.

El prototipo actual utiliza los grupos "Área", "Modalidad", "Tipo de empleo" y "Ordenar por". Dentro de ellos se presentan opciones demostrativas como Tecnología, Administración, Marketing, Ventas, Presencial, Remoto, Híbrido, Tiempo completo, Medio tiempo, Pasantía, Más relevantes y Más recientes. Las opciones pueden ser marcadas y desmarcadas por el usuario, conservándose su estado mientras el programa se encuentra en ejecución. Los conceptos de programación orientada a eventos utilizados fueron los siguientes:

- **Callbacks:** se implementan mediante la función "procedimientoVentana", registrada como procedimiento de ventana de Win32 mediante "lpfnWndProc". Win32 invoca automáticamente este callback cuando se produce un evento y la función delega posteriormente el mensaje recibido en "manejarMensaje()", donde se procesan las diferentes interacciones de la aplicación.
- **Bucles:** la aplicación utiliza el bucle principal de mensajes de Win32 mediante "GetMessage", "TranslateMessage" y "DispatchMessage". Este mecanismo mantiene activa la aplicación, obtiene los eventos generados y los envía al callback correspondiente hasta que se recibe el mensaje de finalización.
- **Eventos Paint:** el repintado principal de la interfaz se realiza mediante el mensaje WM_PAINT, que ejecuta el proceso de dibujo utilizando "BeginPaint" y "EndPaint". Adicionalmente, se utiliza WM_DRAWITEM para realizar el dibujo personalizado del botón de filtros. Cuando una interacción modifica el estado de la interfaz se utiliza "InvalidateRect" para solicitar su actualización visual.
- **Funciones de conversión:** se utilizan principalmente para adaptar las coordenadas reales recibidas desde la ventana al sistema fijo de coordenadas utilizado por la maqueta. De esta forma, los clics pueden seguir correspondiéndose con los elementos visuales, aunque cambie el tamaño de la ventana. Asimismo, el sistema gráfico incluye conversiones de posiciones, dimensiones y colores para su utilización mediante Win32/GDI. La interfaz se construye mediante diferentes componentes reutilizables. Entre ellos se encuentran elementos destinados al encabezado, buscador, tarjetas de

ofertas, etiquetas y botón de filtros, mientras que el panel de filtros mantiene de forma separada su propio estado y comportamiento. También se implementa un sistema de escalado que parte de una maqueta base de 1920 × 920 píxeles y adapta sus dimensiones y posiciones al tamaño real de la ventana, conservando las proporciones de la interfaz. Cabe aclarar que los archivos correspondientes al antiguo renderizador por consola, junto con las funciones de conversión y modelos asociados a esa implementación, se conservan únicamente como evidencia de una versión anterior y no forman parte de la implementación gráfica actual. La entrega final utiliza la interfaz Win32/GDI.³

En el código fuente de la versión actual quedan explícitamente señalados los usos de los conceptos de Callbacks, bucle despachador de eventos, eventos Paint y funciones de conversión, permitiendo relacionar directamente la implementación realizada con los requisitos técnicos establecidos por la consigna.

Conclusión

A lo largo del desarrollo de este proyecto fue posible analizar una problemática concreta de usabilidad presente en el portal de empleos seleccionado, y plantear una alternativa orientada a mejorar la organización y accesibilidad de su sistema de filtros. A partir de este análisis, se desarrolló un prototipo funcional en C++ que permite representar de manera práctica las mejoras propuestas, manteniendo el alcance reducido y demostrativo establecido para el trabajo.

La implementación realizada permitió integrar los principales conceptos de programación orientada a eventos estudiados durante la materia, incluyendo callbacks, bucles despachadores de eventos, eventos Paint y funciones de conversión. Mediante su uso conjunto fue posible construir una interfaz gráfica capaz de responder a las acciones del usuario, actualizar dinámicamente su representación visual y adaptar sus elementos a las dimensiones de la ventana.

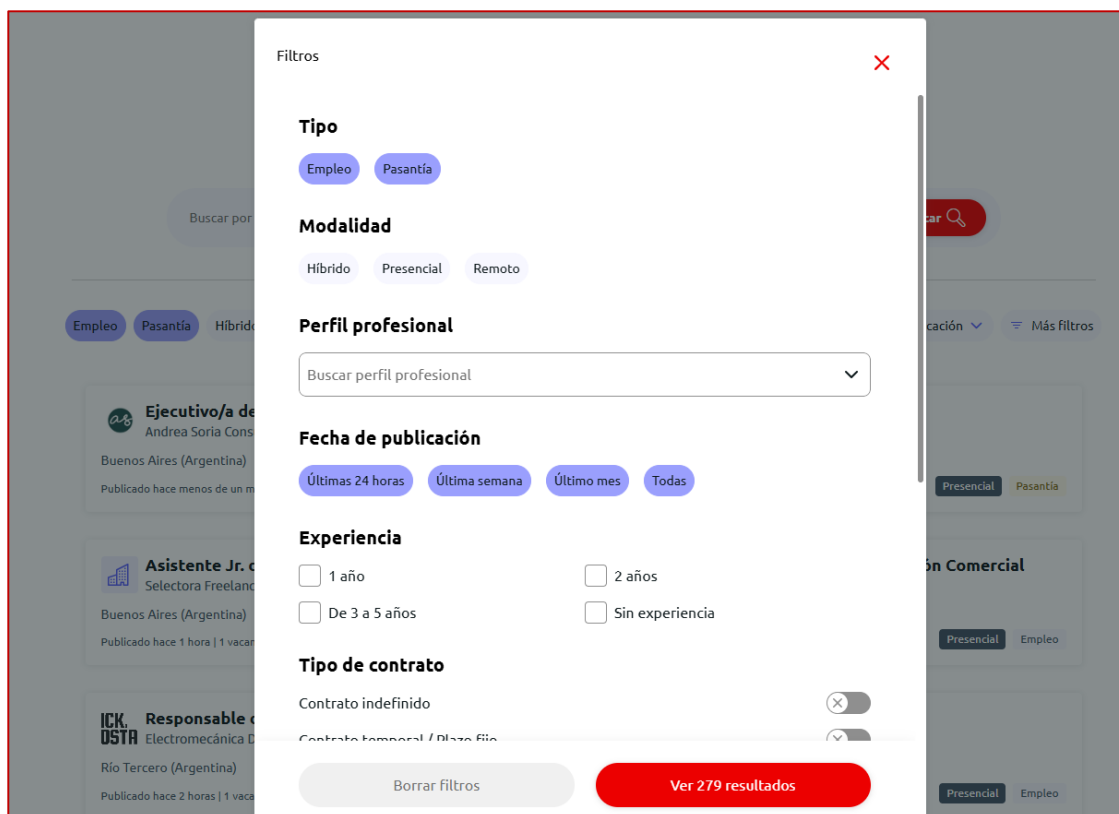
Si bien el prototipo no implementa la aplicación efectiva de los filtros sobre las ofertas laborales, sí permite abrir y cerrar el panel, seleccionar y deseleccionar sus diferentes opciones y conservar visualmente su estado durante la ejecución. De esta manera, se logró demostrar la interacción propuesta sin exceder los límites definidos para el proyecto.

En conclusión, el desarrollo permitió aplicar los contenidos técnicos abordados durante la materia dentro de una problemática concreta de diseño de interfaces, demostrando cómo la programación orientada a eventos puede utilizarse para desarrollar mecanismos de interacción más claros, organizados y comprensibles para el usuario.

³ Ver anexo 3.

Anexos

Anexo 1: Página web de jobboard.universia.net, con muestra del sistema de filtros al cual se hace referencia:



Anexo 2: Mock-Up inicial del sistema de filtros desarrollado:


UNIVERSIDAD
Blas Pascal

Lanzate a por el trabajo que buscas

Ofertas de trabajo para empezar tu carrera profesional o darle un giro de 180 grados.

 **FILTROS**

 **Pasantía Administrativa - Part Time (estudiantes...**
Spinlock NMR
Cordoba (Argentina)
Publicado hace 3 días | 1 vacante

Presencial

Pasantia


UNIVERSIDAD
Blas Pascal

Universisidad Blas Pascal
Publicado hace 7 días | 1 vacante

Presencial

Pasantia

Filtros

X

Opciones disponibles

Area

☐ Tecnologia

☐ Administracion

☐ Marketing

☐ Ventas

Modalidad

☐ Presencial

☐ Remoto

☐ Hibrido

Tipo de empleo

☐ Tiempo completo

☐ Medio tiempo

☐ Pasantia

Ordenar por

☐ Mas relevantes

☐ Mas recientes

Cerrar

10

Anexo 3: Implementaciones de los conceptos de programación orientada a eventos aprendidos:

Implementación de “Callbacks” de la API WIN32, encontrada en el archivo “Aplicacion.cpp”, en líneas 170 – 195.

```
// =====  
// PUNTO DE LA CONSIGNA: CALLBACK  
// Win32 invoca automaticamente esta funcion cada vez que ocurre un evento.  
// El callback recupera la instancia de Aplicacion y delega el mensaje en  
// manejarMensaje(), donde se atienden clics, Paint y cierre de la ventana.  
// Se registra anteriormente mediante wc.lpfWndProc.  
// =====  
static LRESULT CALLBACK procedimientoVentana(HWND hwnd, UINT mensaje, WPARAM wParam, LPARAM lParam) {  
    Aplicacion* app = NULL;  
  
    if (mensaje == WM_NCCREATE) {  
        CREATESTRUCT* datosCreacion = (CREATESTRUCT*)lParam;  
        app = (Aplicacion*)datosCreacion->lpCreateParams;  
        SetWindowLongPtr(hwnd, GWLP_USERDATA, (LONG_PTR)app);  
    } else {  
        app = (Aplicacion*)GetWindowLongPtr(hwnd, GWLP_USERDATA);  
    }  
  
    if (app != NULL) {  
        return app->manejarMensaje(hwnd, mensaje, wParam, lParam);  
    }  
  
    return DefWindowProc(hwnd, mensaje, wParam, lParam);  
}
```

Implementación del bucle despachador de eventos encontrada en el archivo “Aplicacion.cpp”, en líneas 21 – 34.

```
// =====  
// PUNTO DE LA CONSIGNA: BUCLE DESPACHADOR DE EVENTOS  
// GetMessage obtiene cada evento; TranslateMessage lo interpreta y  
// DispatchMessage lo envia al callback procedimientoVentana.  
// El bucle mantiene activa la aplicacion hasta recibir WM_QUIT.  
// =====  
while (GetMessage(&mensaje, NULL, 0, 0)) {  
    TranslateMessage(&mensaje);  
    DispatchMessage(&mensaje);  
}  
  
return 0;
```

Implementación de eventos “Paint” en el archivo “Aplicacion.cpp”, en líneas 47 – 64.

```
// =====  
// PUNTO DE LA CONSIGNA: EVENTOS PAINT  
// WM_PAINT solicita repintar la ventana. dibujar() realiza el  
// proceso dentro de BeginPaint/EndPaint usando el estado actual.  
// =====  
case WM_PAINT:  
    dibujar(hwnd);  
    return 0;  
  
case WM_DRAWITEM:  
    // Evento Paint adicional para dibujar manualmente el boton.  
    if (wParam == ID_BOTON_FILTROS) {  
        botonFiltros.dibujar((const DRAWITEMSTRUCT*)lParam);  
        return TRUE;  
    }  
    break;
```

Implementación de funciones de conversión en el archivo “Aplicacion.cpp”, entre las líneas 144 – 170

```
// =====  
// PUNTO DE LA CONSIGNA: FUNCIONES DE CONVERSION  
// Estas funciones convierten las coordenadas reales recibidas desde Win32  
// al sistema de coordenadas fijo del diseno. Asi, los clics coinciden con  
// los elementos visuales aunque cambie el tamaño de la ventana.  
// =====  
int convertirX(LPARAM lParam, HWND hwnd) const {  
    EscalaVentana escala = EscalaVentana::desdeVentana(hwnd);  
    if (escala.ancho == 0) {  
        return 0;  
    }  
  
    int mouseX = GET_X_LPARAM(lParam) - escala.margenX;  
    return (mouseX * DISENO_ANCHO) / escala.ancho;  
}  
  
int convertirY(LPARAM lParam, HWND hwnd) const {  
    EscalaVentana escala = EscalaVentana::desdeVentana(hwnd);  
    if (escala.alto == 0) {  
        return 0;  
    }  
  
    int mouseY = GET_Y_LPARAM(lParam);  
    return (mouseY * DISENO_ALTO) / escala.alto;  
}
```